STGS / PROJET C++ & LINUX

Space Telemetry Ground Station

Station sol simulée · TCP/UDP · protocole binaire · décodage multithread · capture et rejeu

J'ai construit STGS pour travailler sur une chaîne de télémétrie complète en C++20. Le thème spatial fournit un bon terrain de jeu, mais ce qui m'intéresse surtout se passe entre le réseau et les fichiers : retrouver les bonnes trames, refuser des données abîmées et gérer plusieurs threads sans obtenir des résultats différents d'une exécution à l'autre.

Le satellite est simulé. Les sockets, le format binaire, les queues, les threads et les exports sont bien réels dans le code. Il n'y a en revanche ni matériel spatial, ni compatibilité CCSDS, ni qualification mission.

Le trajet d’une trame



Le réseau reconstitue une **trame candidate**. Le codec contrôle ensuite sa structure et son CRC. Les décodeurs peuvent travailler en parallèle ; un writer unique remet les résultats en ordre avant de mettre à jour l’état de la station ou les fichiers.

Ce qu’on peut faire avec le projet

- Générer de la télémétrie et la recevoir en TCP ou UDP sur une machine Linux.
- Enregistrer une capture STGF puis la relire avec exactement les mêmes octets.
- Envoyer des messages texte et des signaux sinusoïdaux bruités ; observer les filtres et métriques.
- Comparer le CSV obtenu avec un seul décodeur ou plusieurs pour vérifier l’ordre des sorties.

Par où commencer dans le dépôt ?

Le **README** donne les commandes de lancement. Pour le cœur du programme, regarder FrameCodec.cpp, NetworkServerTcp.cpp, BlockingQueue.hpp et GroundStationWorkers.cpp. Les notes Markdown du dossier **docs/** complètent ces points.

Les choix qui comptent

1. Un format binaire simple, mais strict

Le protocole STGS v1 contient un en-tête de 23 octets, un payload de 0 à 4 096 octets et un CRC de 4 octets. Les entiers sont encodés en big-endian. Avant de traiter une trame, le codec vérifie les tailles, la version, le CRC et la validité de certains champs. Les erreurs de trame sont renvoyées comme des valeurs typées, pas comme des exceptions.

Élément	Rôle
Magic « STGS » + version	Retrouver les candidats dans le flux et identifier le format
Longueur bornée	Éviter de faire confiance à une taille arbitraire envoyée sur le réseau
CRC-32/ISO-HDLC	Détecter une corruption accidentelle ; ne garantit pas l'identité de l'émetteur
Payload STGA facultatif	Transporter un message ou des échantillons sans changer le format externe

2. TCP et UDP ne se traitent pas de la même façon

TCP livre des octets, pas des trames. Chaque connexion possède donc son extracteur ; il peut conserver une trame incomplète ou en récupérer plusieurs dans un seul `recv()`. La boucle réseau s'appuie sur `poll()` et sur un instantané des clients pour éviter des indices incohérents. UDP conserve les frontières des datagrammes, mais la station vérifie les troncatures avec `MSG_TRUNC`.

3. Décoder en parallèle sans perdre l'ordre

Chaque candidat reçoit un numéro de séquence avant d'entrer dans la file de travail. Les workers décodent sans écrire dans le CSV. Le writer reçoit leurs résultats et attend la prochaine séquence avant de produire une sortie. Ça permet de comparer deux relectures d'une même capture avec 1 et 8 décodeurs : les fichiers obtenus sont identiques.

La file d'entrée et celle des résultats utilisent mutex et conditions variables. À la fermeture, les threads sont réveillés et les éléments restants sont drainés. Le writer, lui, possède une `std::map` de réordonnancement : cette map n'a pas encore de limite propre. C'est un vrai sujet à traiter si l'on veut borner toute la mémoire du pipeline.

4. Rejouer pour comprendre

Le format STGF garde les trames d'origine, CRC compris. Je peux donc isoler une séquence problématique du réseau et la relire sans changer le chemin de décodage. Le simulateur distingue aussi deux phénomènes : bruit ajouté à un signal **avant** CRC et corruption d'une trame **après** CRC.

Reproduire, puis améliorer

Le meilleur point de départ est une capture STGF : on vérifie le traitement du programme sans dépendre d'un port réseau libre. Ensuite, on peut lancer le simulateur et la station sur `127.0.0.1`.

Compilation et test rapide

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j2
ctest --test-dir build --output-on-failure
bash scripts/verify_demo.sh
```

Ce qui a été vérifié le 2 novembre 2026

Contrôle local	Résultat observé
Compilation GCC 14.2 Release	OK, warnings traités comme erreurs
Tests C++	20 cas réussis (1 cible CTest)
Build ASan / UBSan	Tests réussis
Rejeu STGF de 1 000 trames	1 000 décodées, 0 rejet, 1 000 écrites
Export 1 versus 8 workers	CSV identiques octet pour octet

Ces essais ne constituent pas une qualification. Le compte rendu **docs/VERIFICATIONS_2026-11-02.md** précise l'environnement et ce qui n'a pas été relancé, notamment TSan et les campagnes de charge multi-clients.

Les limites que je ne cherche pas à masquer

- Le réordonnancement peut accumuler des résultats si une séquence plus ancienne tarde trop.
- La réception TCP n'impose pas encore de plafond global de clients ni de quota par connexion.
- Le CRC n'apporte pas d'authentification ; la version actuelle n'embarque pas TLS.
- Le benchmark ne mesure ni réception réseau ni écriture disque. Pas de garantie de latence ou de débit en production.
- Les filtres de signal sont pédagogiques et ne forment pas une chaîne radiofréquence.

La suite logique

Si je continue ce projet, la priorité sera la gestion des ressources en charge : fenêtre de séquences en vol, limites côté réseau, puis tests de saturation et fuzzing des parseurs. Pour moi, ce sont des évolutions plus intéressantes que d'ajouter des fonctionnalités sans pouvoir en vérifier les limites.